

Received 22 January 2025, accepted 6 February 2025, date of publication 11 February 2025, date of current version 14 February 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3540450

RESEARCH ARTICLE

Low-Complexity Architecture for High-Speed 50G-PON LDPC Decoder

KON-WOO KWON¹, (Member, IEEE), KWANGOK KIM², KYEONGHWAN DOO²,
HWANSEOK CHUNG^{1,2}, (Senior Member, IEEE), AND JEONG WOO LEE^{1,3}, (Member, IEEE)

¹Department of Computer Engineering, Hongik University, Seoul 04066, South Korea

²Optical Network Laboratory, Network Research Division, Electronics and Telecommunications Research Institute, Daejeon 34129, South Korea

³School of Electrical and Electronics Engineering, Chung-Ang University, Seoul 06974, South Korea

Corresponding author: Jeong Woo Lee (jwlee2@cau.ac.kr)

This work was supported in part by the Institute of Information and Communications Technology Planning and Evaluation (IITP), Information Technology Research Center (ITRC), 20% grant funded by Korean Government [Ministry of Science and ICT (MSIT)] under Grant IITP-2025-RS-2022-00156353; in part by the National Research Foundation of Korea (NRF) grant funded by Korean Government (MSIT) under Grant RS-2024-00340852; in part by IITP grant funded by Korean Government (MSIT), Development of Access Agnostic-Converged Optical Access Technology, under Grant RS-2023-00215959; in part by the BrainLink Program funded by the MSIT through the NRF under Grant RS-2023-00237308; and in part by the NRF Grant funded by Korean Government under Grant NRF-2023R1A2C2006290.

ABSTRACT We propose a hardware architecture for 50G-PON LDPC decoder achieving high throughput and high error correcting capability while maintaining low level of resource utilization and implementation complexity. Our approach employs phased decoding as a key algorithm which effectively balances the competing goals of high throughput and low amount of resource utilization. We also propose a fixed-structured wiring network between variable nodes and check nodes, leveraging the quasi-cyclic property of parity-check matrix to significantly reduce implementation complexity. To further simplify the decoder structure and enhance the throughput, we propose a blockwise column cyclic shift in the parity-check matrix, along with rules for normalizing and quantizing messages generated in decoder. Our design also incorporates a pipelined structure of computation units. By integrating the proposed design schemes, we draw a sophisticated architecture for high-speed and low-complexity LDPC decoder that achieves the seamless decoding throughput of 49.7664 Gbps, even on a single FPGA board.

INDEX TERMS 50G-PON, LDPC decoder, high throughput, low complexity, architecture, FPGA.

I. INTRODUCTION

Over the last two decades, passive optical network (PON) technology has evolved exponentially to support the fast-growing demand for higher bandwidth and faster connectivity from emerging services like ultra-HD video, immersive experiences, 5G wireless transport and cloud computing [1]. Among the latest advances to meet this demand, 50G-PON, short for 50 Gigabits per second PON, is a cutting-edge technology poised to revolutionize fiber-optic broadband networks [2], [3]. 50G-PON operates on the principle of fiber-to-the-premises (FTTP), delivering ultra-fast broadband connectivity directly to end-users' premises with minimal

signal degradation and maximum reliability. With significant improvements in spectral efficiency and network capacity, 50G-PON enables service providers to accommodate the escalating bandwidth demands of both residential and enterprise consumers.

Forward error correction (FEC) scheme plays a crucial role in ensuring reliable communications in G-PON systems by detecting and correcting errors that may occur over optical fibers during data transmission [4]. Reed-Solomon (RS) codes [5], [6] and Bose-Chaudhuri-Hocquenghem (BCH) codes [6] have been widely used in G-PON systems to correct burst errors, which are common in optical communication systems. They offer efficient error correction capabilities with relatively low computational complexity, making them suitable for high-speed data transmission in

The associate editor coordinating the review of this manuscript and approving it for publication was Mohammad Hossein Moaiyeri¹.

G-PON. Recently, low-density parity-check (LDPC) codes are chosen as a FEC scheme in modern PON systems, e.g., IEEE802.3 Ethernet [7] and 50G-PON [3], thanks to their superior error correction capability. LDPC codes are known to achieve near-optimal error correction capabilities with low redundancy by using sufficiently long codeword, making them highly efficient in terms of bandwidth utilization [8], [9], [10]. However, encoding and decoding of long LDPC codes are computationally intensive, requiring efficient algorithm and hardware architecture to be employed in a modem for real-time operation with limited resource usages.

LDPC codes can be represented by a bipartite factor graph composed of variable nodes and check nodes connected by edges [11], [12]. LDPC codes are decoded in an iterative manner, where messages updated in variable nodes are delivered to connected check nodes and messages updated in check nodes are delivered to connected variable nodes. Based on this characteristics, the LDPC decoder can be implemented with a parallel structure, in which message updates in all variable nodes are performed at the same time and message updates in all check nodes are conducted simultaneously. Such a parallel structure approach enables high decoding throughput, but at the expense of significant hardware resource utilization. To alleviate the heavy resource demands, LDPC decoders can be designed for sequential operations, allowing computational units to be reused over multiple time steps, albeit with degraded throughput. Therefore, an appropriate level of parallelization needs to be carefully considered to balance throughput and resource utilization. Layered decoding has been proposed as a viable solution for achieving this balance by enabling a group of nodes to update messages simultaneously [13], [14], [15], [16], [17]. QC-LDPC codes [18], [19], [20], whose parity-check matrix is composed of circulant submatrices, are suitable for layered decoding to be applied to. This implementation technique has been successfully applied to LDPC decoders in various wireless communication systems with throughput requirements in the hundreds of Mbps. Scheduling of message passing in QC-LDPC decoder based on layered decoding has been of great concern in many research works to improve throughput and reduce implementation complexity [13], [14], [15], [16], [17]. However, the LDPC decoder for 50G-PON modem necessitates a more sophisticated architecture, as achieving 50 Gbps throughput requires a higher level of parallelization and resource utilization. Many applications, including G-PON modems, needs to operate within limited computational resources, where the implementation on a field-programmable gate array (FPGA) board is an example. Thus, it is required to design LDPC decoder architectures that achieve high throughput while minimizing hardware resource usage. Although LDPC codes have been incorporated into the 50G-PON standard [3] and many efforts have been made to design high-speed LDPC decoders on FPGA platforms [21], [22], [23], [24], no previous work has demonstrated the implementation of an LDPC decoder on a single FPGA board that achieves

an actual 50 Gbps throughput in a seamless manner. For example, the demonstration in [24] attained 90 Gbps but required 75 FPGA boards.

In this paper, we propose a systematic and efficient mechanism for determining the design parameters of LDPC decoders, aiming to achieve an optimal balance among throughput, error correcting capability, and resource utilization. We also propose a sophisticated architecture of hard-decision LDPC decoder designed to deliver high throughput and low bit error rate (BER) while achieving low levels of implementation complexity and resource utilization. The proposed architecture includes phased decoding, low-complexity fixed-structured wiring networks, pipelined structure as well as normalization and quantization rules for messages exchanged during iterative decoding. By effectively leveraging the quasi-cyclic (QC) property of the LDPC code's parity-check matrix, we design reusable computation units and introduce fixed wiring networks that improve interconnect efficiency. We analyze the performance of the LDPC decoder with our proposed architecture, demonstrating a significant reduction in resource utilization and a notable improvement in throughput compared to conventional designs. Synthesis, implementation, and post-implementation timing simulations confirm that the proposed architecture enables the LDPC decoder to achieve a seamless real-time throughput of 49.7664 Gbps on a single AMD Kintex UltraScale+ XCKU19P FPGA, while utilizing a low amount of computing resources.

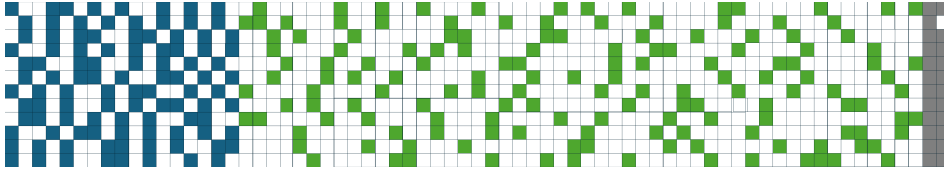
As main contributions,

- Phased decoding with pipelined structures achieves a decoding throughput of 49.7664 Gbps while maintaining a low amount of FPGA resource utilization, requiring 429,785 LUTs and 485,147 flip-flops.
- Fixed-structured wiring network significantly reduces wiring requirements for the LDPC decoder, utilizing only 12.8% of the wiring needed for conventional approaches.
- Normalization and quantization rule for messages improves decoding performance without increasing complexity, providing high error correcting capability.

This paper is organized as follows. In Section II, encoding and decoding procedures of 50G-PON LDPC codes are briefly introduced. In Section III, we propose low-complexity architecture for high-speed LDPC decoder, which includes phased decoding, fixed-structured wiring network, efficient mapping rule for message normalization and quantization, and pipelined structures of computation units. Then, we analyze the pipelined phased decoding in terms of throughput. In Section IV, we present hardware implementation of the proposed LDPC decoder and evaluate the resulting performance in FPGA. Finally, we conclude the paper in Section V.

II. ENCODING AND DECODING OF 50G-PON LDPC CODES

Let $\mathbf{u}$ denote a K -bit information block. Zero-padding bits of length S are appended at the end of $\mathbf{u}$ optionally according

FIGURE 1. Structure of parity-check matrix $\mathbf{H}$.

to the communication scenario resulting in LDPC message bits $\mathbf{u}^* = [\mathbf{u} \mid \mathbf{0}]$ of length $K + S = 14592$. Parity bits $\mathbf{p} = [\mathbf{p}_1 \mid \mathbf{p}_2]$ are generated from $\mathbf{u}^*$, where $\mathbf{p}_1$ and $\mathbf{p}_2$ are of length $M = 2688$ and $P = 384$, respectively, and thus the length of $\mathbf{p}$ is 3072 bits. Then, a mother codeword $\mathbf{v}$ is obtained in a systematic form as $\mathbf{v} = [\mathbf{u}^* \mid \mathbf{p}]$, where the length of $\mathbf{v}$ is $14592 + 3072 = 17664$ bits. Note that $\mathbf{v}$ satisfies a parity-check equation $\mathbf{H}\mathbf{v}^T = \mathbf{0}$, where $\mathbf{H}$ is a parity-check matrix of LDPC codes with a dimension of $(M + P) \times (K + S + M + P) = 3072 \times 17664$. Note that $\mathbf{H}$ has a sparse structure as shown in Fig. 1, which can be represented by 12×69 blocks of circulant submatrices. Each submatrix has a dimension of 256×256 , where white blocks represent zero matrices and colored blocks represent an identity matrix or its cyclic-shifted versions. The number of cyclic-shifts applied to individual 256×256 identity submatrices are defined in the standard [3]. In this manner, the parity-check matrix $\mathbf{H}$ has a QC structure. Puncturing is applied to remove $\mathbf{p}_2$ from $\mathbf{p}$, and shortening is optionally applied to remove zero-padding bits from $\mathbf{v}$, by which LDPC codeword $\mathbf{c} = [\mathbf{u} \mid \mathbf{p}_1]$ is finally obtained. The length of resulting codeword $\mathbf{c}$ is $N = K + M = 14592 + 2688 = 17280$ bits. In case that no zero-padding bits are applied, i.e., $S = 0$, we obtain (17280, 14592) LDPC codes with the code rate of $R_c = K/N = 14592/17280 = 0.8444$.

LDPC codeword $\mathbf{c}$ is transmitted over an optical channel and received as the sequence of binary bits 1 or 0 after passing a photo detector and a demodulator, which is input to the hard-decision LDPC decoder. We may regard the overall channel from the transmitter to the input of LDPC decoder as a binary symmetric channel (BSC) with a cross-over probability ϵ . Since a mother codeword $\mathbf{v}$ is generated from $\mathbf{u}^*$ based on a parity-check matrix as explained above, the function of LDPC decoder is obtaining estimates for $\mathbf{u}^*$ from the contaminated version of $\mathbf{v}$. Decoding starts from assigning channel log-likelihood ratio (LLR) of each entry v , defined as $L_c(v) = \log \frac{\Pr(v=0)}{\Pr(v=1)}$. Since zero-padding bits in $\mathbf{u}^*$ have a fixed value 0, the corresponding channel LLRs are assigned maximum values, where zero-padding bits are shortened and not actually transmitted. Part of parity bits $\mathbf{p}_2$ are punctured and not transmitted, so LLRs of $\mathbf{p}_2$ are assigned 0. LLRs of $\mathbf{u}$ and $\mathbf{p}_1$ are determined as $L_c(v) = (-1)^{\tilde{v}} \log \frac{1-\epsilon}{\epsilon}$, where $\tilde{v}$ denotes the contaminated value of v representing entries of $\mathbf{u}$ and $\mathbf{p}_1$.

LDPC codes can be represented by a bipartite factor graph composed of check nodes and variable nodes connected through edges [11], [12]. Each variable node and check node corresponds to each column and row of parity-check matrix

Algorithm 1 Normalized Min-Sum Algorithm (NMSA)

Channel LLR Calculation: For given channel output $\tilde{v}_j, j = 0, \dots, K + S + M + P$,

$$L_c(v_j) = \begin{cases} (-1)^{\tilde{v}_j} \log \frac{1-\epsilon}{\epsilon}, & \text{if } 0 \leq j < K \text{ or } K+S \leq j < K+S+M, \\ \max L, & \text{if } K \leq j < K+S, \\ 0, & \text{if } j \geq K+S+M, \end{cases} \quad (1)$$

CVM Initialization:

$$L(r_{ij}) = 0, \quad \text{for all } i \text{ and } j \in V_i. \quad (2)$$

Counter Initialization: CNT = 0.

while CNT < N_I **do**

VCM Update:

$$L(Q_j) = L_c(v_j) + \sum_{i \in C_j} L(r_{ij}), \quad \text{for all } j$$

$$L(q_{ji}) = L(Q_j) - L(r_{ij}), \quad \text{for all } j \text{ and } i \in C_j. \quad (3)$$

CVM Update:

$$L(r_{ij}) = \gamma \times \prod_{j' \in V_i \setminus j} \alpha_{ji'} \times \min_{j' \in V_i \setminus j} \beta_{ji'}, \quad \text{for all } i \text{ and } j \in V_i. \quad (4)$$

where $\alpha_{ji} = \text{sign}(L(q_{ji}))$ and $\beta_{ji} = |L(q_{ji})|$.

CNT = CNT + 1

end while

Decision:

$$\hat{u}_j = \begin{cases} 1, & \text{if } L(Q_j) < 0 \\ 0, & \text{else.} \end{cases} \quad \text{for } 0 \leq j < K. \quad (5)$$

$\mathbf{H}$, respectively. LDPC decoding is performed iteratively by updating LLR messages at variable nodes and check nodes. Let h_{ij} be the (i, j) -th entry of $\mathbf{H}$, where i and j denote row index and column index, respectively, with $0 \leq i < M + P$ and $0 \leq j < K + S + M + P$. It is clear that i and j also denote check node index and variable node index, respectively. Let $L(r_{ij})$ be the LLR check-to-variable message (CVM) sent from the i -th check node to the j -th variable node. We also let $L(q_{ji})$ be the LLR variable-to-check message (VCM) sent from the j -th variable node to the i -th check node. We use a normalized min-sum algorithm (NMSA) for high-speed LDPC decoding with low-complexity resulting in reasonably high error correcting capability [27]. The NMSA is summarized in Algorithm 1. Note that $\max L$ is the maximum value of channel LLR, γ is a normalization factor, C_j is a set of check nodes connected to the variable node j , V_i denotes a set of variable nodes connected to the check node i and $V_i \setminus j$ denotes a set of variable nodes except j connected to the check node i . Considering the structure of bipartite factor graph and parity-check matrix of LDPC codes, it is clear that

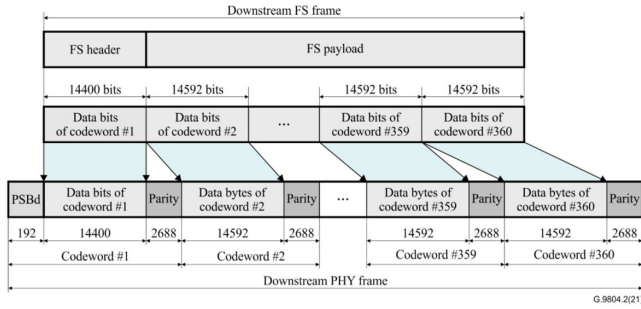


FIGURE 2. Downstream PHY frames in 50G-PON [3].

V_i is also the set of column indices j for which $h_{ij} = 1$ in a given row i of $\mathbf{H}$ and C_j is the set of row indices i for which $h_{ij} = 1$ in a given column j .

III. LOW-COMPLEXITY ARCHITECTURE FOR HIGH-SPEED LDPC DECODER

50G-PON standard [3] requires the downstream throughput of 49.7664 Gbps while upstream throughput is chosen among 12.4416 Gbps, 24.8832 Gbps and 49.7664 Gbps. We aim to design LDPC decoder architecture achieving 49.7664 Gbps downstream throughput, which is the most challenging scenario in 50G-PON communications. Fig. 2 shows downstream PHY frame in 50G-PON data transmission. As depicted in Fig. 2, the downstream PHY frame is composed of 360 LDPC codewords and is transmitted for 125 μsec to achieve 49.7664 Gbps throughput, where each LDPC codeword is 17280 bit-length. It follows that the decoding process for each 17280-bit codeword must begin every 0.34722 μsec . To enable seamless decoding of continuously received codewords, multiple decoding submodules can be employed in the hardware, with each submodule decoding codewords alternately. Let Z , F and N_I denote the number of submodules implemented in the decoder, the clock frequency in Hz and the number of decoding iterations, respectively. Then, in order to achieve the decoding throughput of 49.7664 Gbps using N_I iterations at F Hz, each submodule must begin a new decoding iteration every $0.34722 \times 10^{-6} \times F \times Z/N_I$ cycles. The reduced resource usage of each decoding submodule allows for the integration of more submodules in the decoder hardware, which in turn increases throughput. With more submodules, additional decoding iterations can be performed, improving BER. Thus, our design begins with developing a low-complexity architecture for decoding submodules, followed by selecting proper values for Z and N_I to achieve target throughput and BER by using the allowed amount of total resource utilization.

A. PHASED DECODING

LDPC decoding submodule consists of variable node units (VNUs) and check node units (CNUs) connected via wiring networks. Each CNU corresponds to a row of $\mathbf{H}$. Neighboring T columns of $\mathbf{H}$ are grouped, with a VNU responsible for updating the VCMs along these columns, where $T \geq 1$ and

$\text{mod}(256, T) = 0$. As a result, for a given $\mathbf{H}$ with the dimension of 3072×17664 , 3072 CNUs and $T' = 17664/T$ VNUs are defined as depicted in Fig. 3.

VCMs associated with columns located at the same position in each column group are updated simultaneously. In other words, for a given time period t , $0 \leq t < T$, VCMs associated with columns with indices $t, t + T, t + 2T, \dots, t + (T' - 1)T$ are updated concurrently. We refer to such a time period as a *phase*. CNU stores the minimum and the second minimum absolute values as well as the product of sign bits obtained at the previous phase. Then, in the next phase, CNU compares newly incoming VCMs and stored values, then updates the minimum and the second minimum absolute values. Sign bits of newly incoming VCMs are multiplied to the stored sign-bit-product, by which the product of sign bits is updated. The minimum, the second minimum absolute values as well as the product of sign bits are combined into a compressed CVM (CCVM), where CCVMs updated at the final phase are delivered to VNUs. In VNUs, prior to updating VCMs, the minimum and the second minimum absolute values as well as the product of sign bits of VCMs are recovered from CCVM. A pair of column-wise VCM updates and row-wise CCVM updates composes one phase, and T phases compose one decoding iteration. The aforementioned decoding procedure will be called a *phased decoding*, where the detailed process is summarized in Algorithm 2 and described by an illustrative example in Fig. 4. In Algorithm 2, we use $\min_1(i)$ and $\min_2(i)$ to represent the minimum and the second minimum absolute values of VCM on the i -th row of $\mathbf{H}$. We also use $sb(i) = \pm 1$ as the product of sign bits of VCMs on the i -th row of $\mathbf{H}$ and define M as an arbitrarily large number. Note that $V_i(t)$ denotes a set of columns j , which are considered during the phase t , having $h_{ij} = 1$ in a given row i of $\mathbf{H}$.

Phased decoding defines the level of parallelization of LDPC decoder and plays a crucial role in balancing throughput, resource utilization, and wiring overhead. The number of phases, T , must be carefully determined to meet the performance requirements of the decoder. When $T = 1$, the LDPC decoding submodule is implemented in a fully parallel structure, maximizing throughput but requiring significant computational resources. Such a design is often impractical for resource-constrained environments. Layered decoding, widely adopted in practical LDPC decoder hardware [13], [14], [15], offers an efficient alternative by reducing resource utilization through lower levels of parallelization. However, layered decoding relies on sequential operations of decoding units, inherently limiting throughput. As a result, layered decoding is unsuitable for high-speed applications such as 50G-PON LDPC decoder. For example, a previous implementation in [24] achieved a total throughput of 90 Gbps but required 75 FPGA boards, yielding just 1.2 Gbps per FPGA.

To address these limitations, we adopt phased decoding with an appropriately chosen T , which offers a partially parallel structure that balances high throughput and resource efficiency. By grouping columns based on the QC property of

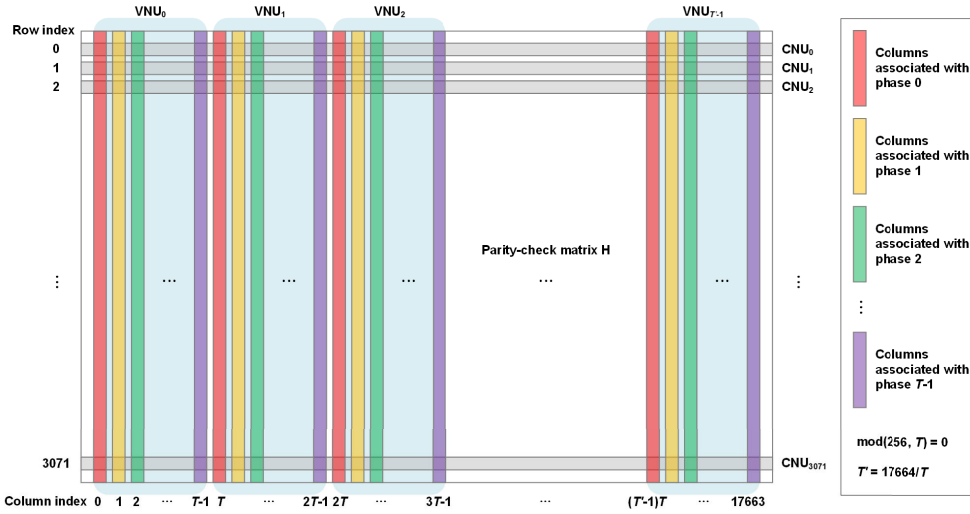


FIGURE 3. Illustration of VNUs and CNUs associated with the parity-check matrix.

Algorithm 2 Summary of Phased Decoding With NMSA

```

1: Compute  $L_c(v_j)$  and initialize  $L(r_{ij}) \leftarrow 0$  for all  $j$  and  $i \in C_j$ 
2: Initialize  $CNT \leftarrow 0$ 
3: while  $CNT < N_I$  do
4:   Initialize  $\min_1(i) = \min_2(i) \leftarrow M$  and  $sb(i) \leftarrow 1$  for all  $i$ 
5:   for  $t = 0$  to  $T - 1$  do
6:     for all  $0 \leq m < T'$  do
7:        $j' = mT + t$ 
8:        $L(Q_{j'}) \leftarrow L_c(v_{j'}) + \sum_{i \in C_{j'}} L(r_{ij'})$ 
9:       for all  $i \in C_{j'}$  do
10:         $L(q_{ji}) \leftarrow L(Q_{j'}) - L(r_{ij'})$ 
11:         $\alpha_{ji} \leftarrow \text{sign}(L(q_{ji}))$  and  $\beta_{ji} \leftarrow |L(q_{ji})|$ 
12:      end for
13:    end for
14:    for all  $i$  do
15:      for all  $j'' \in V_i(t)$  do
16:        if  $\beta_{ji} \leq \min_1(i)$  then
17:           $\min_2(i) \leftarrow \min_1(i)$  and  $\min_1(i) \leftarrow \beta_{ji}$ 
18:        else if  $\min_1(i) < \beta_{ji} < \min_2(i)$  then
19:           $\min_2(i) \leftarrow \beta_{ji}$ 
20:        end if
21:      end for
22:       $sb(i) \leftarrow sb(i) \times \alpha_{ji}$ 
23:    end for
24:  end for
25:  for all  $i$  do
26:    for all  $j \in V_i$  do
27:      if  $\beta_{ji} = \min_1(i)$  then
28:         $L(r_{ij}) \leftarrow \gamma \times \min_2(i) \times \alpha_{ji} \times sb(i)$ 
29:      else
30:         $L(r_{ij}) \leftarrow \gamma \times \min_1(i) \times \alpha_{ji} \times sb(i)$ 
31:      end if
32:    end for
33:  end for
34:   $CNT \leftarrow CNT + 1$ 
35: end while

```

the parity-check matrix, phased decoding enables the reuse of VNUs across multiple phases. This approach reduces the required VNUs to $1/T$ of the total, significantly lowering resource utilization. Additionally, when combined with our proposed fixed wiring scheme, phased decoding minimizes wiring overhead by reducing the total number of connections needed for routing. Phased decoding also facilitates effective

pipelining of operations, enabling higher throughput without increasing resource complexity.

As will be discussed in subsequent sections, phased decoding, in combination with fixed wiring networks, blockwise column cyclic shift of $\mathbf{H}$, LLR normalization and quantization, and pipelining, achieves an effective tradeoff, delivering 49.776 Gbps using a single FPGA resource.

B. LOW-COMPLEXITY WIRING

In LDPC decoder implementation, all pairs of column (or variable node) j and row (or check node) i of $\mathbf{H}$ having $h_{ij} = 1$ must be wired to exchange VCM and CVM correctly. Even in phased decoding, pairs of variable node and check node, which need to be connected, vary every phase so the wiring complexity is basically equivalent to that of fully parallel decoding. However, wiring all individual pairs of variable node and check node results in an extremely high level of complexity because 70400 ones exist in $\mathbf{H}$ of 50G-PON LDPC codes. Implementing too many wires may result in long wiring paths between basic components so that severe congestion and timing violation are highly probable especially when the decoder is implemented on FPGA. To alleviate such a wiring complexity, we design a fixed wiring network by utilizing the QC property of parity-check matrix and the reusable property of VNU and CNU in phased decoding.

For a general description, we define $W \times W$ as the dimension of circulant submatrices of parity-check matrix. Let $CCVM_i$ denote $CCVM$ corresponding to the i -th row obtained at the final phase of the previous decoding iteration. Suppose a certain input port of VNU receives $CCVM_i$ at the phase t , where $t < T - 2$. Due to the QC property of $\mathbf{H}$, this input port receives $CCVM_{i'}$ at the next phase $t + 1$, where

$$i' = W \times \lfloor i/W \rfloor + \text{mod}(i + 1, W). \quad (6)$$

This property is observed in an illustrative example given in Fig. 4, where $W = 3$ and $T = 3$. Let us consider

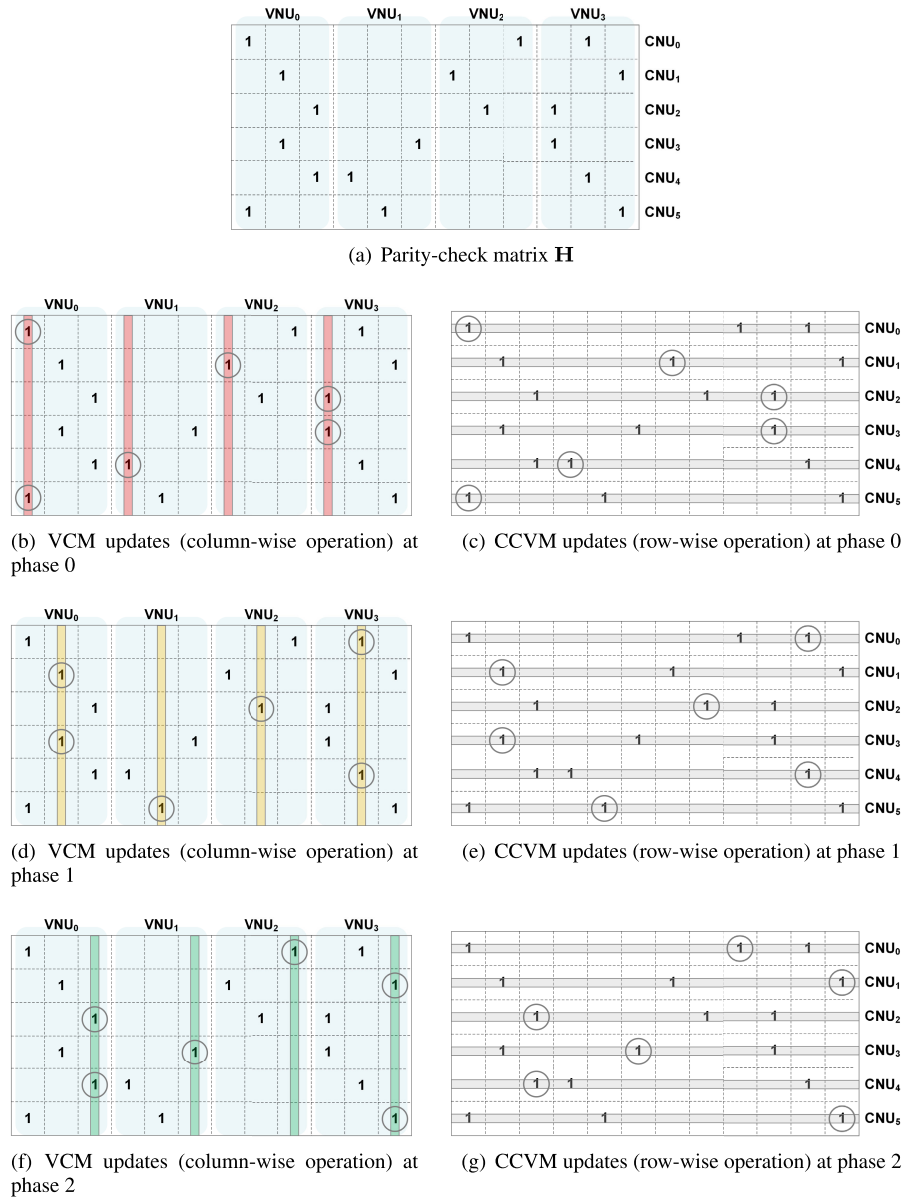


FIGURE 4. Illustration of VNU and CNU operations in phased decoding with $T = 3$. CCVMs obtained at phase 2 are used for updating VCMs in the next decoding iteration.

VNU₀ whose leftmost column has two ones on the rows 0 and 5. This implies that VNU₀ needs two input ports, each of which receives CCVM₀ and CCVM₅, respectively, at phase $t = 0$. By using (6), we know that $i = 0$ is mapped to $i' = 3 \times \lfloor 0/3 \rfloor + \text{mod}(1, 3) = 1$ and $i = 5$ is mapped to $i' = 3 \times \lfloor 5/3 \rfloor + \text{mod}(6, 3) = 3$. Thus, it is expected that VNU₀ receives two inputs CCVM₁ and CCVM₃ at phase $t = 1$, which matches the actual observation. In a similar manner, $i = 1$ is mapped to $i' = 3 \times \lfloor 1/3 \rfloor + \text{mod}(2, 3) = 2$ and $i = 3$ is mapped to $i' = 3 \times \lfloor 3/3 \rfloor + \text{mod}(4, 3) = 4$. Thus, it is expected that VNU₀ receives two inputs CCVM₂ and CCVM₄ at phase $t = 2$, which matches the observation.

It is notable that the relationship in (6) also applies to a blockwise cyclic shift register, which is formed by $3072/W$

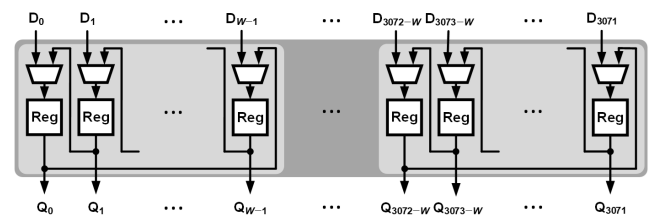


FIGURE 5. Structure of blockwise cyclic shift register.

blocks of W -tap cyclic shift registers as shown in Fig. 5. The first term $W \times \lfloor i/W \rfloor$ in the right hand side of (6) determines the index of W -tap register block while the second term

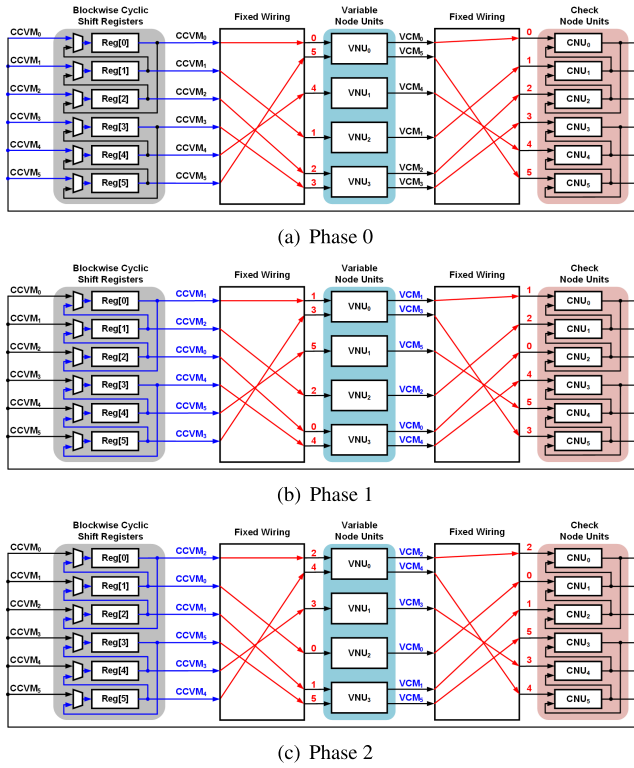


FIGURE 6. Illustrative example of fixed wiring between VNUs and CNUs using blockwise cyclic shift registers, where $W = 3$ and $T = 3$.

$\text{mod}(i + 1, W)$ determines the position in the specific register block. Let us consider a vector $\{a_0, a_1, \dots, a_{3071}\}$ stored in a blockwise cyclic shift register in this order. Suppose a_i is stored in a register i at a certain time instance and we apply a blockwise cyclic shift operation. Then, at the next time instance, the entry stored in the register i is found to be $a_{i'}$, where i' is given by (6). By using this property, we may deliver CCVMs to target variable nodes by using a low-complexity fixed-structured wiring network. Wiring from VNUs to CNUs can also be implemented through a blockwise cyclic shift register based on the QC property of $\mathbf{H}$. At every phase, each CNU updates CCVM and passes this to the succeeding CNU, where CCVM is updated by comparing incoming VCMs and stored CCVM fed from the preceding CNU at the previous phase. Fig. 6 is provided as an illustrative example of the aforementioned property. First, let us consider the phase 0 depicted in Fig. 6(a). VNU₀ receives CCVM₀ and CCVM₅ and generates VCMs sent to CNU₀ and CNU₅ to be used for updating CCVM₀ and CCVM₅, respectively. VNU₁ receives CCVM₄ and generates VCM sent to CNU₄ to be used for updating CCVM₄. VNU₂ receives CCVM₁ and obtains VCM sent to CNU₁ to be used for updating CCVM₁. VNU₃ receives CCVM₂ and CCVM₃ and generates VCMs sent to CNU₂ and CNU₃ to be used for updating CCVM₂ and CCVM₃, respectively. At phase 1, entities in the register block and the CNU block are blockwise cyclic shifted as shown in Fig. 6(b) and the same procedure as in phase 0 is conducted. CCVM₁ and

CCVM₃ are delivered to VNU₀ to obtain VCMs sent to CNU₀ and CNU₅ to be used for updating CCVM₁ and CCVM₃, respectively. CCVM₅ is fed to VNU₁ and the corresponding VCM is sent to CNU₄ to update CCVM₅, while CCVM₂ is fed to VNU₂ and the obtained VCM is sent to CNU₁ to update CCVM₂. CCVM₀ and CCVM₄ are delivered to VNU₃ to generate VCMs sent to CNU₂ and CNU₃ for the purpose of updating CCVM₀ and CCVM₄, respectively. As shown in Fig. 6(c), we have the same operation at phase 2. It is obvious that flows of VCMs and CCVMs between variable nodes and check nodes of $\mathbf{H}$ given in Fig. 4 can be correctly executed by using a fixed wiring network provided in Fig. 6. It is notable that CCVM corresponding to a certain check node (or row of $\mathbf{H}$) is updated by multiple CNUs based on QC property. For example, CCVM₀ is updated in CNU₀, CNU₂ and CNU₁ at phase 0, phase 1 and phase 2, respectively. Based on QC property of $\mathbf{H}$, wiring from VNUs to CNUs and from CNUs to VNUs can be implemented in a fixed structure by using a blockwise cyclic shift register. All individual pairs of variable nodes and check nodes do not need to be wired physically. This contribution can reduce significantly the wiring complexity in LDPC decoder hardware.

Using phased decoding and the fixed wiring network introduced above, we propose an LDPC decoder architecture for each decoding submodule, as shown in Fig. 7. To reduce the number of wires connecting the block interleaver, located in the upper area of Fig. 7, and the VNUs, we in practice calculate channel LLRs directly inside the VNUs based on the received codeword, minimizing external wiring needs.

To further reduce wiring complexity between VNUs and CNUs, we propose blockwise cyclic shifts of columns in $\mathbf{H}$ as explained below. The number of VNU inputs is determined by the weight of the columns associated with each VNU. It is clear from Fig. 1 and Fig. 3 that all columns belonging to a given VNU have an identical weight. Thus, $17 \times 256/T$ VNUs have 6 inputs, $50 \times 256/T$ VNUs have 3 inputs, $256/T$ VNUs have 12 inputs and $256/T$ VNUs have 11 inputs. In order to reduce the implementation complexity of decoder, it would be better to maintain the regularity of the structure of CNU. Note that the maximum number of inputs to CNU i at phase t is equal to the maximum number of ones existing in the intersections of the i -th row and columns of $\mathbf{H}$ belonging to phase t . Thus, we define the number of inputs to CNU as the maximum cardinality of $V_i(t)$ considering all i and t . We had better lower the number of inputs to CNU for the purpose of reducing the implementation complexity. For this purpose, we apply cyclic shifts to 256 columns in each column block of $\mathbf{H}$ to minimize the number of inputs to CNU. Note that received codeword bits need to be reordered accordingly so that channel LLRs are computed correctly in VNUs. By elaborative search, we found the maximum numbers of inputs to individual CNUs for various T , which are listed in Table 1.

Beyond optimizing the wiring between CNUs and VNUs, we further simplify the architecture by reducing the bit width

TABLE 1. The maximum number of inputs to each CNU for various phase numbers, T .

T	1	2	4	8	16	32	64	128	256
Max Number of Inputs	23	18	12	7	4	2	2	2	2

of the CCVM format. In many conventional approaches [28], [29], the index of VCM with the minimum absolute value is stored in the CCVM, and variable nodes use this index when executing the operation in (3) to obtain VCMs. In contrast, our method directly compares the minimum absolute value recovered from the CCVM with the absolute values of VCM to perform (3) in VNUs. Our approach eliminates the need to store the index of VCM having the minimum absolute value, simplifying the CCVM format, which would otherwise require an additional 5 bits to represent the position of the minimum absolute value among 22 or 23 VCMs. This reduction in bit width significantly decreases wiring complexity.

To estimate the total wiring requirements for the LDPC decoding submodule using the proposed low-complexity wiring network, we first examine a basic wiring scheme based on barrel shifter structures [25], [26]. In this configuration, $70400/T$ of T -to-1 multiplexers are required to route messages from variable nodes to check nodes. Assuming a quantized bit-width of b for VCMs, the total number of wires required for this operation can be expressed as $\{b \times 70400(T+1)/T\}$, excluding the wires used to control the multiplexers. Here, each multiplexer contributes $b \times (T+1)$ wires, consisting of $b \times T$ input wires and b output wires. Additionally, 3072 check nodes update and send their CCVMs to associated registers at each decoding iteration. Since CCVMs are represented by $2b-1$ bits, this operation requires $\{(2b-1) \times 3072\}$ wires. To send CCVMs back from the registers to variable nodes, $70400/T$ of T -to-1 multiplexers are required, resulting in additional wiring requirement of $\{(2b-1) \times 70400 \times (T+1)/T\}$. Furthermore, the decoding submodule must deliver received codewords to variable nodes, necessitating additional $\{17764 \times (T+1)/T\}$ wires. For example, when $T = 32$ and $b = 4$, the total number of wires required to implement the LDPC decoding submodule using this basic wiring scheme is 838424, without considering miscellaneous wires, which have a negligible impact on the overall count.

In contrast, our low-complexity approach, which utilizes a fixed wiring network with a blockwise shift register, significantly reduces wiring overhead. By distributing the workload across $17664/T$ VNUs and maintaining 3072 CNUs, we leverage the average weight of a single column in $\mathbf{H}$, which is $70400/17664$. Then, the total number of inputs to VNUs is $70400/17664 \times 17664/T = 70400/T$, which also equals the total number of outputs from VNUs. From CNUs to the blockwise shift register, $3072 \times (2b-1)$ wires are required. Since the blockwise shift register includes 2-to-1 multiplexers, this wiring expands to $(2b-1) \times 3072 \times 3$,

accounting for two inputs and one output per multiplexer. For messages routed from registers to variable nodes, the fixed wiring scheme requires only $(2b-1) \times 70400/T$ wires, which is $(T+1)$ times fewer than the basic scheme. Additionally, $17764 \times (T+1)/T$ wires are needed to route codewords from the block interleaver to VNUs. Combining all components, the total number of wiring requirement for the proposed scheme is $b \times 70400/T + (2b-1) \times 3072 \times 3 + (2b-1) \times 70400/T + 17764 \times (T+1)/T$. For $T = 32$ and $b = 4$, the ratio of wiring required by the proposed wiring scheme to that of the basic wiring scheme is 0.128, as shown in Table 2. It is clear that with higher number of phases, we can observe more significant merit of using the proposed wiring scheme instead of a basic wiring scheme.

C. NORMALIZATION AND QUANTIZATION OF LLR

As one of ingredients for reducing the decoder hardware complexity, the bit width of LLR messages needs to be determined as the minimum value achieving a decoding performance comparable to that obtained with the soft-valued LLR. The most critical factor for determining the bit width of LLR is the performance degradation incurred by normalizing CVM so that an appropriate mapping from the original CVM to the normalized CVM needs to be defined.

By floating-point simulations, we find that the normalization factor $\gamma = 0.75$ applied to the hard-decision NMSA-based iterative decoding results in the lowest BER over a wide range of cross-over probability in channel, i.e., input BER. Then, we consider a primitive mapping rule converting a soft value to the nearest fixed-point value. It is observed from Fig. 8 that at least five bits are required to represent LLR messages, one of which is used as a sign bit and four of which are used for absolute value in an integer form, in order to avoid significant decoding performance degradation in case that a primitive mapping rule is applied to normalization. Then, we found that there exists a mapping rule for normalization by which the minimum bit width of LLR can be reduced to four without resulting in a noticeable decoding performance degradation, which is inferred from Fig. 8. Based on this observation, we apply the proposed mapping rule for normalization and quantize LLR as four bits, one of which is a sign bit and three of which are used to represent absolute value in an integer form. By applying the proposed mapping rule for normalization, a three-bit absolute value of LLR, $\mathbf{a} = (a[2] \ a[1] \ a[0])$, is mapped to a new three-bit vector $\mathbf{a}'$ as

$$\mathbf{a}' = (\mathbf{a}_{(10)} - (a[2] \vee a[1]))_{(2)}, \quad (7)$$

where $(\cdot)_{(2)}$ denotes a conversion of a decimal number to a three-bit binary number, $\mathbf{a}_{(10)}$ denotes a decimal number representing $\mathbf{a}$, and $\vee$ denotes a logical OR operation. The mapping of each three-bit vector to a normalized one is listed in Table 3.

There exist various approximated Min-Sum algorithms (MSAs), among which normalized min-sum algorithm (NMSA) [27], offset min-sum algorithm (OMSA) [27] and

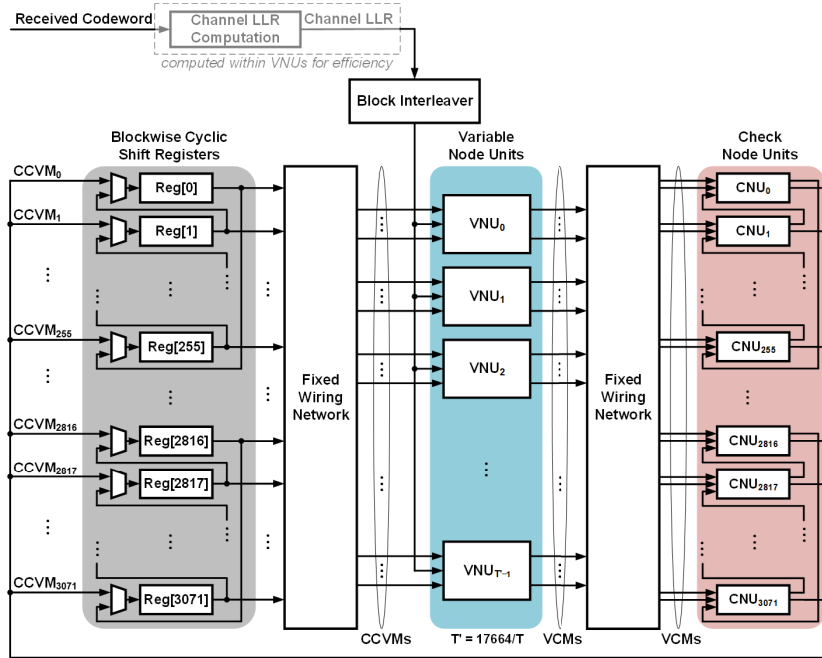


FIGURE 7. Hardware architecture of the LDPC decoding submodule.

TABLE 2. Comparison of wiring requirements.

	Barrel shifter scheme ($= B$)	Fixed wiring scheme ($= F$)	Ratio ($= F/B$)
Messages from variable nodes to check nodes	$b \times 70400(T+1)/T$	$b \times 70400/T$	$1/(T+1)$
Messages from check nodes to registers	$(2b-1) \times 3072$	$(2b-1) \times 3072 \times 3$	3
Messages from registers to variable nodes	$(2b-1) \times 70400 \times (T+1)/T$	$(2b-1) \times 70400/T$	$1/(T+1)$
Codewords from interleaver to variable nodes	$17764 \times (T+1)/T$	$17764 \times (T+1)/T$	1
Total number of wires when $T = 32$ and $b = 4$	838424	107032	0.128

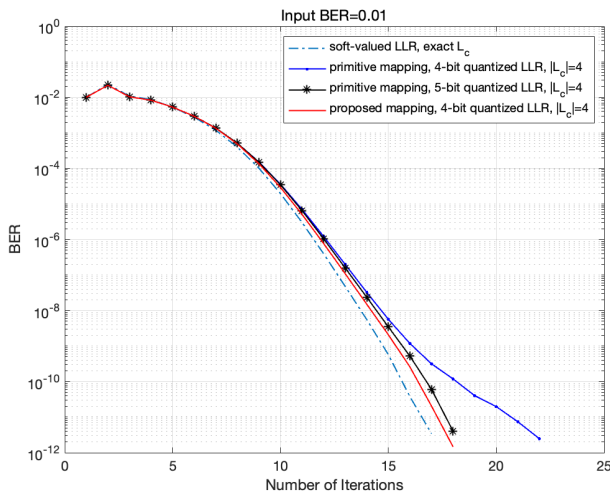


FIGURE 8. BER performances obtained with various bit-widths of LLRs and mapping rules for normalizing CVM, where the absolute value of channel LLR is fixed as 4. BER obtained with soft-valued LLR with exact channel LLR is also plotted.

normalized offset min-sum algorithm (NOMSA) [30] are well known examples. It is known that NOMSA achieves the lowest BER among these approximated MSAs at the cost of

TABLE 3. Three-bit quantization of normalized absolute values.

Original	000	001	010	011	100	101	110	111
Normalized	000	001	001	010	011	100	101	110

extra complexity and resource utilization. In case of using low bit-width quantization, we cannot apply exact values of normalization factor and offset factor so that NOMSA and NOMSA cannot be clearly discriminated. It follows that by defining an appropriate mapping rule, we can obtain better performance than NOMSA, where the resulting performance may approach to that of NOMSA. It is obvious that a primitive mapping rule is a natural approach to implement NOMSA. In the proposed mapping rule, the results of a primitive mapping are additionally modified, which may have an implicit effect of additionally applying offset factors to NOMSA. Thus, the proposed mapping rule improves the decoding performance over the primitive mapping. The performance improvement is significant at low BER when low bit-width quantization is considered.

We also found that channel LLRs, denoted as L_c , do not need to be computed exactly according to the input BER, when quantized LLRs are used in a decoding process. As a result, we determine channel LLRs to be constant irrespective

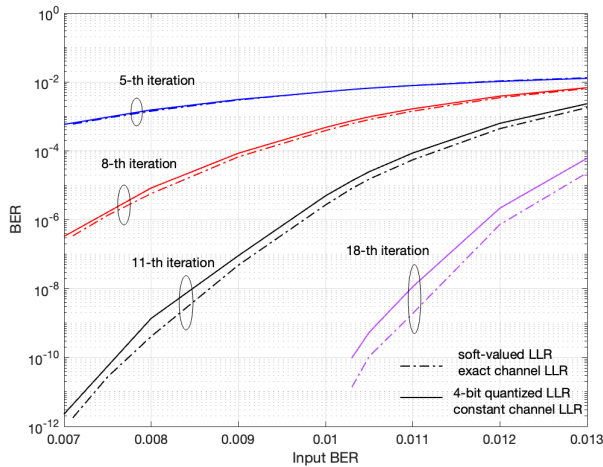


FIGURE 9. BER performances obtained with 4-bit quantized LLR, the proposed normalization mapping rule and a constant channel LLR, where BERs obtained by floating point simulation with exact channel LLR are also plotted.

of the input BER. This may reduce the decoder complexity because estimating the input BER requires a high level of complexity. We empirically found that the appropriate absolute value of channel LLR is 4, or 100 in a binary vector, which results in a lower BER at high numbers of iterations. Fig. 9 shows that BER performance obtained with 4-bit quantized LLR, the proposed normalization mapping rule and a constant channel LLR is comparable to BER obtained by using soft-valued LLR and exactly computed channel LLR.

D. PIPELINED STRUCTURE

The throughput of decoding submodule can be improved by incorporating pipelined structures for both VNU and CNU. In this subsection, we present the pipelined structure of VNU and CNU and discuss pipelined phased decoding with respect to throughput and resource utilization.

1) VNU STRUCTURE

Fig. 10 illustrates an example of a 3-stage pipelined VNU structure, which processes three incoming CCVMs along with their associated codeword bit concurrently. In the first stage, CVMs are recovered from CCVMs while propagating the codeword bit. To recover each CVM, the minimum absolute value in CCVM is compared to VCM obtained in the previous iteration. If the values differ, the CVM's magnitude is set to the normalized minimum value based on our specified normalization formula in (7). Otherwise, the magnitude is set to the normalized second minimum. The CVM's sign is computed by XORing the sign from CCVM with the sign of VCM from the previous iteration.

In the second pipeline stage of VNU, the 4-bit quantized LLR is derived from the propagated 1-bit codeword. Three CVMs are then summed with the derived LLR to obtain $L(Q_j)$ as described in Algorithm 2. All of three CVMs and $L(Q_j)$ are passed through flip-flops to be synchronized and prepared for propagation to the third pipeline stage.

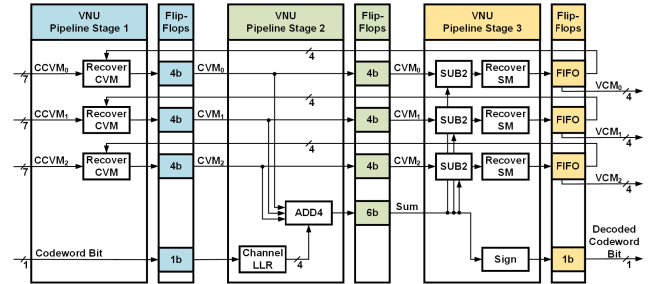


FIGURE 10. Hardware architecture of three-stage-pipeline-structured VNU in LDPC decoder, where three CVMs coming into VNU are considered.

In the third pipeline stage of VNU, individual VCMs are updated by subtracting the corresponding CVMs from $L(Q_j)$. During this process, VCMs are represented in two's complement format for ease of arithmetic operations. After the subtraction, they are converted to sign-and-magnitude format to facilitate CVM updates by CNUs. The converted VCMs are then propagated to CNUs and inserted into a first-in-first-out (FIFO) queue for use in the first pipeline stage of the next iteration. Additionally, the third stage extracts the sign from $L(Q_j)$, which eventually corresponds to the decoded codeword bit after the target number of iterations.

2) CNU STRUCTURE

In this subsection, we present an example of a 2-stage pipelined CNU structure, as shown in Fig. 11, which processes two incoming VCMs, each represented in sign-and-magnitude format. In the first stage of CNU, the magnitudes of two VCMs are compared, and if necessary, swapped to ensure the larger magnitude is placed in the lower position. Additionally, the signs of two VCMs are XORed to be used in the next stage.

In the second pipeline stage of CNU, a pseudo rank order filter processes two pre-sorted pairs of VCM magnitudes in parallel, selecting the minimum and second minimum values. As shown in Fig. 11, one pair is sourced from the previous stage within the same CNU, while the other pair comes from a neighboring CNU. This configuration is based on the wiring equipped with a blockwise cyclic shift register between VNUs and CNUs, which leverages the QC property of $\mathbf{H}$ in our fixed-structure wiring network. Additionally, the sign bit from the previous stage is XORed with the sign bit from the neighboring CNU. The result at the final phase, combining the minimum and the second minimum absolute values as well as the XORed sign bit, forms a CCVM that will be used in the next iteration.

3) PIPELINED PHASED DECODING

Note that the number of pipelined stages for VNU (or CNU) is defined according to the number of inputs to VNU (or CNU) and the clock frequency. To maintain the regularity of architecture, the number of pipelined stages for all VNUs (or CNUs) need to be defined identically, and thus the maximum number of inputs to VNU (or CNU) is of concern. Since

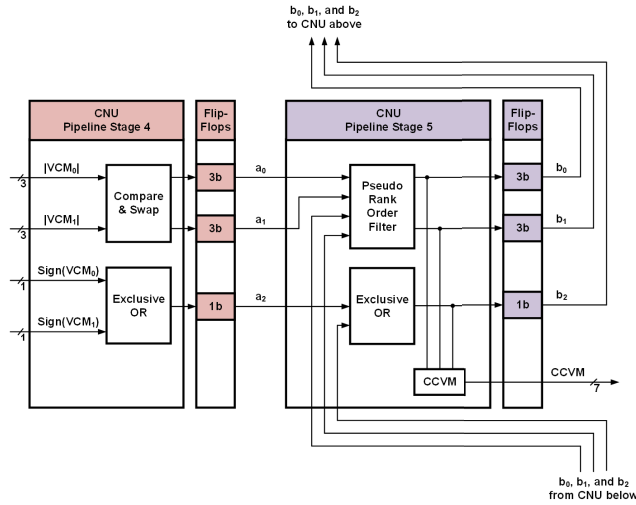


FIGURE 11. Hardware architecture of two-stage-pipeline-structured CNU in LDPC decoder, where two VCMs coming into CNU are considered.

columns belonging to a certain VNU have the same weight, the maximum number of inputs to VNU does not depend on T . On the other hand, the number of inputs to CNU is determined by T as shown in Table 1. Consequently, the number of pipelined stages for VNU is represented by a function of F while the number of pipelined stages for CNU is represented by a function of T and F .

Let n_c be the number of inputs to CNU, and let $s_v(F)$ and $s_c(n_c, F)$ denote the number of pipelined stages required for VNU and CNU, respectively, for a given F , where $s_v(F)$ and $s_c(n_c, F)$ are nondecreasing functions with respect to F for a given n_c . We also let $c(T)$ denote the maximum number of inputs to CNU in phased decoding with T phases, which is listed in Table 1. To maintain the regularity of architecture, we implement all CNUs having the identical number of inputs as $n_c = c(T)$. The operation of VNU is just a summation of input messages, so $s_v(F)$ does not depend on the number of inputs. Then, one phase, which is defined as a sequence of VNU and CNU operations, can be completed by $s_v(F) + s_c(c(T), F)$ stages. By using pipelined structures for VNU and CNU, we can perform multiple phases in a staggered manner with a timing offset of a duration for a stage as depicted in Fig. 12, where $s_v(F) = 3$ and $s_c(c(T), F) = 2$ are considered, and one stage requires one cycle to be completed. Consequently, the total number of cycles required to complete one decoding iteration is obtained by

$$\begin{aligned} & \text{number of cycles per iteration} \\ &= \text{number of phases} + \text{number of stages per phase} - 1 \\ &= T + s_v(F) + s_c(c(T), F) - 1 \quad [\text{cycles}]. \end{aligned} \quad (8)$$

Without using a pipelined structure, the number of cycles per iteration in phased decoding would be $T \times (s_v(F) + s_c(c(T), F))$. The reduction of cycle numbers per iteration resulting from using pipelined structure is noticeable. In case of using fully parallel decoding, the number of cycles per iteration would be $s_v(F) + s_c(n_c = 23, F)$, where

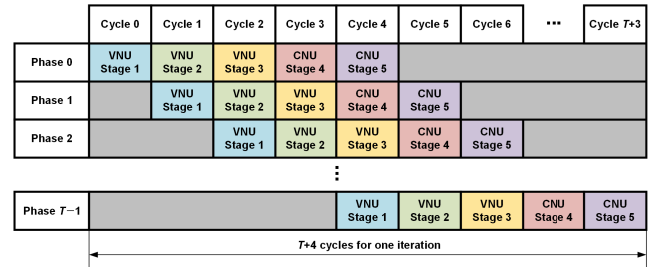


FIGURE 12. Concept of phase and required clock cycles for each decoding iteration by considering pipelined structure, where 3 stages and 2 stages are considered for pipelining VNU and CNU, respectively.

$s_c(23, F) \geq s_c(c(T), F)$ for $T \neq 1$. Let us consider layered decoding as another reference for comparison. Suppose each layer uses parallel decoding to attain the highest throughput at the cost of using high amount of resources. Then, the number of cycles required for each layer operation is $s_v(F) + s_c(23, F)$. In case of 50G-PON LDPC codes, there exist 12 layers, so the total number of cycles per iteration would be $12 \times (s_v(F) + s_c(23, F))$. Layered decoding enables fast convergence of LLR messages, so we suppose only a half number of iterations are required to obtain the same BER as phased decoding. Considering this factor, it is fair to compare the number of cycles per half iteration in layered decoding, $6 \times (s_v(F) + s_c(23, F))$, and the number of cycles per iteration in phased decoding, $T + s_v(F) + s_c(c(T), F) - 1$. As the clock frequency F gets higher, the number of pipelined stages for CNU with 23 inputs becomes relatively much higher than that for CNU with a small number of inputs caused by high T . Thus, it is expected that phased decoding outperforms layered decoding in view of throughput and resource utilization when high clock frequency is used to implement high speed LDPC decoder.

IV. HARDWARE IMPLEMENTATION OF LDPC DECODER

A. DESIGN PARAMETERS

As introduced earlier, each decoding iteration must be completed within $0.34722 \times 10^{-6} \times F \times Z/N_I$ cycles. By considering (8), we must satisfy the condition of

$$T + s_v(F) + s_c(c(T), F) - 1 \leq 0.34722 \times 10^{-6} \times F \times Z/N_I \quad (9)$$

for hardware implementation of LDPC decoder based on phased decoding. To achieve higher error correcting capability, we had better use a higher number of iterations N_I at the cost of a probable throughput degradation. To improve the throughput, we need to increase the level of parallelization of decoder structure reflected by a lower T and a higher Z . However, in case that the amount of resources is limited, the level of lowering T and increasing Z is limited. Thus, we must determine design parameters N_I , T and Z properly such that the condition (9) is met, and high throughput as well as high error correction capability is achieved with a limited amount of resource utilization.

TABLE 4. Feature summary of AMD Kintex UltraScale+ XCKU19P [31].

Resource Unit	System Logic Cells	CLB Flip Flops	CLB LUTs
Number	1,842,750	1,684,800	842,400

TABLE 5. Cyclic shift numbers for 69 column blocks resulting in CNU having $c(T = 32) = 2$.

Column Block	0	1	2	3	4	5	6	7
Shift Number	0	0	0	62	62	186	155	217
Column Block	8	9	10	11	12	13	14	15
Shift Number	93	93	124	248	0	54	186	85
Column Block	16	17	18	19	20	21	22	23
Shift Number	186	124	155	0	116	248	124	23
Column Block	24	25	26	27	28	29	30	31
Shift Number	178	248	217	23	62	124	23	116
Column Block	32	33	34	35	36	37	38	39
Shift Number	147	217	0	85	209	23	62	240
Column Block	40	41	42	43	44	45	46	47
Shift Number	178	217	15	108	155	147	85	77
Column Block	48	49	50	51	52	53	54	55
Shift Number	248	209	85	108	46	162	69	15
Column Block	56	57	58	59	60	61	62	63
Shift Number	7	178	15	209	38	40	131	23
Column Block	64	65	66	67	68	-	-	-
Shift Number	170	147	46	0	0	-	-	-

We consider AMD Kintex UltraScale+ XCKU19P as a target FPGA, whose operating clock frequency is $F = 388.8$ MHz and numbers of some important resource units are listed in Table 4. Then, the condition (9) is rewritten as $T + s_v(F) + s_c(c(T), F) - 1 \leq 135 \times Z/N_I$. By elaboration, we determine the number of decoding submodules as $Z = 3$ and the number of phases as $T = 32$, which are maximum and minimum values for Z and T , respectively, guaranteeing the utilization of limited amount of resources in a target FPGA board. We found that $c(T = 32) = 2$ as listed in Table 1 and CNU can be implemented with two pipelined stages, i.e., $s_c(2, F = 388.8 \times 10^6) = 2$. Note that $c(T = 32) = 2$ is obtained by applying the blockwise cyclic shift to columns of $\mathbf{H}$, where 256 columns in each column block are cyclic shifted by numbers listed in Table 5. We also found that VNU can be implemented with three pipelined stages, i.e., $s_v(F = 388.8 \times 10^6) = 3$. It follows that the condition (9) becomes $T + 4 \leq 135 \times Z/N_I$ and the maximum number of iterations satisfying this condition with $T = 32$ and $Z = 3$ is $N_I = 11$. As a result, phased decoding can be implemented on the target FPGA with $17664/32 = 552$ VNUs and 3072 CNUs. Note that $\text{VNU}_0, \dots, \text{VNU}_{135}$ have six inputs, $\text{VNU}_{136}, \dots, \text{VNU}_{535}$ have three inputs, and $\text{VNU}_{536}, \dots, \text{VNU}_{543}$ have twelve inputs, and $\text{VNU}_{544}, \dots, \text{VNU}_{551}$ have eleven inputs.

The LDPC decoder implemented with the proposed architecture on a target FPGA board, AMD Kintex UltraScale+ XCKU19P, can achieve the throughput of 49.7664 GHz at the operating clock frequency of 388.8 MHz and obtain the BER of 10^{-12} with 11 decoding iterations at the input BER of 7×10^{-3} .

Let us compare the throughput of phased decoding with that of layered decoding, where fully parallel decoding is assumed for each layer in layered decoding to achieve the

highest throughput. In case of fully parallel decoding with $n_c = 23$, we found that $s_c(n_c = 23, F = 388.8 \times 10^6) = 5$. Thus, in layered decoding, each layer operation requires $s_v(F) + s_c(23, F) = 3 + 5 = 8$ stages, where there exist 12 layers. Suppose layered decoding requires 5 iterations to obtain BER comparable to that obtained by phased decoding with 11 iterations. Then, to achieve a certain BER obtained by phased decoding with $36 \times 11 = 393$ cycles, layered decoding requires $5 \times 8 \times 12 = 480$ cycles. Even the maximum throughput achievable by layered decoding with the highest parallelization level is lower than the throughput achieved by phased decoding with lower amount of resource utilization. It is obvious that the throughput of layered decoding would be further degraded if it is implemented with a limited amount of resources and thus partially parallel decoding is applied for each layer operation.

B. EVALUATION

To evaluate the performance of the proposed architecture, we implemented the LDPC decoder on AMD Kintex UltraScale+ XCKU19P FPGA. The decoder processes continuously incoming punctured codewords, each of which has a length of 17280 bits. Each codeword is divided into 128-bit segments over 135 cycles, while operating at a frequency of 388.8 MHz. This configuration corresponds to a throughput of 49.7664 Gbps, derived from processing 388.8×10^6 segments of 128 bits per second.

With design parameters of $Z = 3$ and $T = 32$, we obtain $s_v(F) = 3$ and $s_c(c(T), F) = 2$ with $F = 388.8$ MHz and $N_I = 11$ on the target FPGA. Each of three decoding submodules processes a newly incoming 135-cycle codeword every 405 cycles, with decoding distributed across three submodules, as shown in Fig. 13. The phased decoding approach completes each decoding iteration in 36 cycles, determined by $T + s_v(F) + s_c(c(T), F) - 1$. With 11 decoding iterations, the total decoding time for a single codeword is 396 cycles. This ensures seamless decoding, as the total decoding time remains within 405 cycles available for processing each incoming codeword.

Table 6 summarizes the key parameters and results of these implementations. The implementation of the proposed LDPC decoder architecture utilizes 429,785 LUTs and 485,147 flip-flops on the FPGA. Despite the high resource utilization rate, the low-complexity fixed wiring between VNUs and CNUs effectively prevents routing congestion, which is typically associated with large resource usage. In addition, the fixed wiring network, combined with blockwise column cyclic shift of $\mathbf{H}$, LLR quantization and pipelining, enables the decoder to operate seamlessly at a target clock frequency of 388.8 MHz.

For a comparative analysis, Table 6 shows different designs from [23] and [24], both of which are also designed for the identical 50G-PON standard with a mother code of 17664-bit length and 57/69 code rate. The layered decoding approach in [23] requires 96 cycles per iteration, substantially greater than the 36 cycles in our phased decoding. This

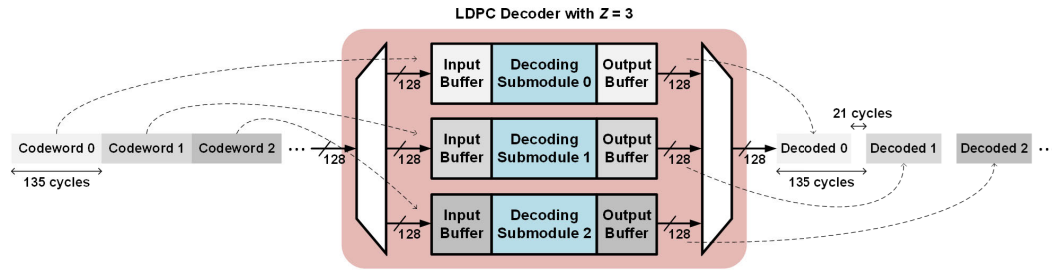


FIGURE 13. LDPC decoder with 3 decoding submodules.

TABLE 6. Comparison of LDPC Decoder Implementations.

Parameter	Proposed	[23]	[24]
Code Family	50G-PON	50G-PON	50G-PON
Mother Code Length	17,664	17,664	17,664
Mother Code Rate	57 / 69	57 / 69	57 / 69
Decoding Algorithm	NMSA	NMSA	NMSA
Decoding Architecture	Phased Decoding	Layered Decoding	Layered Decoding
Cycle Count per Iteration	36	96	N/A
Number of Iterations	11	12	12
FPGA Family	AMD Kintex UltraScale+	AMD Virtex-7	AMD Virtex UltraScale
FPGA Part Name	XCKU19P	XC7VX485T	XCVU125
LUT Utilization	429,785	144,086	537,504
Flip-Flop Utilization	485,147	76,012	675,854
Operating Frequency (MHz)	388.8	100	75
Throughput (Gbps)	49.766	1.235	1.2

increased cycle count is attributed to the sequential nature of the layered decoding, which introduces an extra factor of 12, as the twelve layers of $\mathbf{H}$ are processed in series. In addition, the layered architecture requires an 8 stages per layer, mainly due to the high number of inputs to each CNU, which further slows down the processing. Operating at a clock frequency of 100 MHz, the decoder in [23] achieves a throughput of 1.235 Gbps, compared to the 49.766 Gbps throughput of our design. Similarly, the decoder in [24] also employs a layered architecture but operates at an even lower clock frequency of 75 MHz, resulting in a throughput of 1.2 Gbps. Both designs are limited by the inherent nature of layered decoding, including the high number of cycle count per iteration and the high number of inputs to CNUs, which constrain their overall throughput. In contrast, the phased decoding approach in our design can balance throughput and resource utilization efficiency, making it a viable solution for the challenging requirements of the 50G-PON standard.

V. CONCLUSION

We designed a hardware architecture for a high-speed and low-complexity hard-decision LDPC decoder tailored for 50G-PON modems. As key components of the LDPC decoder, we proposed phased decoding, fixed-structured low-complexity wiring networks, pipelined structures for VNU and CNU as well as normalization and quantization rules for LLR messages exchanged during iterative decoding. Additionally, we provided an efficient framework for determining the design parameters of LDPC decoder to achieve a balanced performance across throughput, error correcting capability, and resource utilization.

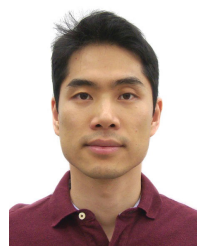
In phased decoding, each group of columns in parity-check matrix are processed simultaneously in each phase without being segmented into layers, which results in higher throughput than layered decoding. Since groups of columns are defined by considering QC structure of parity-check matrix, VNUs and CNUs can be reused multiple times resulting in low amount of resource utilization. By implementing VNU and CNU in a pipelined structure under phased decoding framework, we can run multiple phases in a staggered manner with one clock cycle offset and thus can improve significantly the decoding throughput. Based on QC property of parity-check matrix, the wiring network between computing units can be implemented in a fixed-structure, which reduces tremendously the wiring complexity of LDPC decoder. Applying blockwise cyclic shift to columns of parity-check matrix offers extra complexity reduction by reducing the number of inputs to each CNU. The proposed mapping rule applies a normalization factor with a rounding condition, which corresponds to NMSA, and then modifies the result additionally. This additional modification offers an implicit effect of NOMSA and results in enhanced BER performance.

The proposed architecture significantly reduces both the computation resources and wiring complexity required to achieve high throughput and high error correcting capability. The proposed LDPC decoder outperforms conventional LDPC decoders in view of throughput while maintaining low resource utilization. Implemented on a single FPGA, our architecture achieves a seamless actual throughput of 49.7664 Gbps, marking a significant advancement compared with previous works.

Phased decoding, fixed-structured wiring network and pipelined structure for VNU and CNU can be directly applied to all variations of MSA, e.g., MSA [32], OMSA, NOMSA and min-sum-correction algorithm (MSCA) [33], for decoding QC-LDPC codes without modification. The mapping rule for LLR normalization and quantization needs to be modified depending on what kind of approximated method is used.

REFERENCES

- [1] K. Kim, K.-H. Doo, H. H. Lee, S. Kim, H. Park, J.-Y. Oh, and H. S. Chung, "High speed and low latency passive optical network for 5G wireless systems," *J. Lightw. Technol.*, vol. 37, no. 12, pp. 2873–2882, Jun. 2019.
- [2] "White paper on 50G-PON technology, V2.0," ZTE, White Paper, 2020. Accessed: Feb. 11, 2025. [Online]. Available: https://www.zte.com.cn/content/dam/zte-site/res-www-zte-com-cn/mediares/zte/global/product_images/fm_pictures/others/White%20Paper%20on%2050G%20PON%20Technology%20V2.0_EN.pdf
- [3] *G.9804.3: 50-Gigabit-Capable Passive Optical Networks (50G-PON): Physical Media Dependent (PMD) Layer Specification*. Accessed: Sep. 2021. [Online]. Available: <https://www.itu.int/rec/T-REC-G.9804.3-202109-I>
- [4] L. Schmalen, A. J. D. L. van Wijngaarden, and S. Ten Brink, "Forward error correction in optical core and optical access networks," *Bell Labs Tech. J.*, vol. 18, no. 3, pp. 39–66, Dec. 2013.
- [5] T. Buerner, R. Dohmen, A. Zottmann, M. Saeger, and A. J. van Wijngaarden, "On a high-speed Reed–Solomon coded architecture for 43 Gb/s optical transmission systems," in *Proc. Int. Conf. Microelectron.*, Nis, Serbia, May 2004, pp. 743–746.
- [6] *G.984.3: Gigabit-capable Passive Optical Networks (G-PON): Transmission Convergence Layer Specification*. Accessed: Jan. 2014. [Online]. Available: <https://www.itu.int/rec/>
- [7] *IEEE Standard for Ethernet—Amendment 9: Physical Layer Specifications and Management Parameters for 25 Gb/s and 50 Gb/s Passive Optical Networks*, IEEE IEEE Standard 802.3ca, 2020.
- [8] R. G. Gallager, *Low-Density Parity-Check Codes*. Cambridge, MA, USA: MIT Press, 1963.
- [9] T. J. Richardson and R. L. Urbanke, "The capacity of low-density parity-check codes under message-passing decoding," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 599–618, Feb. 2001.
- [10] T. J. Richardson, M. A. Shokrollahi, and R. L. Urbanke, "Design of capacity-approaching irregular low-density parity-check codes," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 619–637, Feb. 2001.
- [11] F. R. Kschischang, B. J. Frey, and H.-A. Loeliger, "Factor graphs and the sum-product algorithm," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 498–519, Feb. 2001.
- [12] G. D. Forney, "Codes on graphs: Normal realizations," *IEEE Trans. Inf. Theory*, vol. 47, no. 2, pp. 520–548, Feb. 2001.
- [13] E. Sharon, S. Litsyn, and J. Goldberger, "An efficient message-passing schedule for LDPC decoding," in *Proc. 23rd IEEE Conv. Electr. Electron. Engineers*, Tel Aviv-Yafo, Israel, Nov. 2004, pp. 223–226.
- [14] D. Hecovar, "A reduced complexity decoder architecture via layered decoding of LDPC codes," in *Proc. IEEE Workshop Signal Process. Syst.*, Austin, TX, USA, Oct. 2004, pp. 107–112.
- [15] P. Radosavljevic, A. de Baynast, and J. R. Cavallaro, "Optimized message passing schedules for LDPC decoding," in *Proc. 39th Asilomar Conf. Signals, Syst. Comput.*, Pacific Grove, CA, USA, 2005, pp. 591–595.
- [16] J. Zhang and M. P. C. Fossorier, "Shuffled iterative decoding," *IEEE Trans. Commun.*, vol. 53, no. 2, pp. 209–213, Feb. 2005.
- [17] C.-W. Sham, X. Chen, W. M. Tam, Y. Zhao, and F. C. M. Lau, "A layered QC-LDPC decoder architecture for high speed communication system," in *Proc. IEEE Asia-Pacific Conf. Circuits Syst.*, Kaohsiung, Taiwan, Dec. 2012, pp. 475–478.
- [18] Y. Yu Tai, L. Lan, L. Zeng, S. Lin, and K. A. S. Abdel-Ghaffar, "Algebraic construction of quasi-cyclic LDPC codes for the AWGN and erasure channels," *IEEE Trans. Commun.*, vol. 54, no. 10, pp. 1765–1774, Oct. 2006.
- [19] S. Myung and K. Yang, "A combining method of quasi-cyclic LDPC codes by the Chinese remainder theorem," *IEEE Commun. Lett.*, vol. 9, no. 9, pp. 823–825, Sep. 2005.
- [20] L. Lan, L. Zeng, Y. Y. Tai, L. Chen, S. Lin, and K. Abdel-Ghaffar, "Construction of quasi-cyclic LDPC codes for AWGN and binary erasure channels: A finite field approach," *IEEE Trans. Inf. Theory*, vol. 53, no. 7, pp. 2429–2458, Jul. 2007.
- [21] M. Yang, Z. Pan, and I. B. Djordjevic, "FPGA-based burst-error performance analysis and optimization of regular and irregular SD-LDPC codes for 50G-PON and beyond," *Opt. Exp.*, vol. 31, no. 6, pp. 10936–10946, Mar. 2023.
- [22] M. Yang, L. Li, X. Liu, and I. B. Djordjevic, "Real-time verification of soft-decision LDPC coding for burst mode upstream reception in 50G-PON," *J. Lightw. Technol.*, vol. 38, no. 7, pp. 1693–1701, Apr. 2020.
- [23] J. Wang, J. Yang, G. Zhang, X. Zeng, and Y. Chen, "Efficient layered parallel architecture and application for large matrix LDPC decoder," *Electronics*, vol. 12, no. 18, p. 3784, Sep. 2023.
- [24] W. Wang, K. Tao, W. Qian, Y. Cai, M. Lei, X. Zhou, H.-C. Chien, J. Liang, S. Zhang, and Z. Liu, "Real-time FPGA verification for 25G-PON and 50G-PON LDPC codes," in *Proc. Conf. Lasers Electro-Optics (CLEO)*, San Jose, CA, USA, May 2020, pp. 1–2.
- [25] E. Boutillon and H. Harb, "Extended barrel-shifter for versatile QC-LDPC decoders," *IEEE Wireless Commun. Lett.*, vol. 9, no. 5, pp. 643–647, May 2020.
- [26] B. L. T. Edara, T. G. N. Aishwarya, S. R. Patlolla, and M. O. V. P. Kumar, "Implementation of LDPC decoder using barrel shifter," in *Proc. 3rd Int. Conf. Appl. Artif. Intell. Comput. (ICAAIC)*, Tamil Nadu, India, Jun. 2024, pp. 1731–1737.
- [27] J. Chen, A. Dholakia, E. Eleftheriou, M. P. C. Fossorier, and X.-Y. Hu, "Reduced-complexity decoding of LDPC codes," *IEEE Trans. Commun.*, vol. 53, no. 8, pp. 1288–1299, Aug. 2005.
- [28] M. Cocco, J. Dielissen, M. Heijligers, A. Hekstra, and J. Huisken, "A scalable architecture for LDPC decoding," in *Proc. Design, Autom. Test Eur. Conf. Exhib.*, Paris, France, 2004, pp. 88–93.
- [29] Z. Wang and Z. Cui, "A memory efficient partially parallel decoder architecture for QC-LDPC codes," in *Proc. 39th Asilomar Conf. Signals, Syst. Comput.*, Pacific Grove, CA, USA, 2005, pp. 729–733.
- [30] M. Shaghaghghi, Y. L. Guan, K. Cai, and Z. Qin, "Combined normalized and offset min-sum decoding over partial response channels," in *Proc. 7th Int. Conf. Inf. Commun. Signal Process. (ICICSP)*, Macau, China, Dec. 2009, pp. 1–4.
- [31] *AMD UltraScale+ FPGAs Product Selection Guide*. Accessed: Jul. 2024. [Online]. Available: <https://docs.amd.com/v/u/en-US/ultrascale-plus-fpga-product-selection-guide>
- [32] M. P. C. Fossorier, M. Mihaljevic, and H. Imai, "Reduced complexity iterative decoding of low-density parity check codes based on belief propagation," *IEEE Trans. Commun.*, vol. 47, no. 5, pp. 673–680, May 1999.
- [33] G. Wang, N. T. D. Linh, and G. Rugumira, "Improved min-sum-plus-correction-factor algorithm using stopping node for LDPC decoding," in *Proc. IEEE Int. Symp. Signal Process. Inf. Technol. (ISSPIT)*, Ho Chi Minh City, Viet Nam, Dec. 2012, pp. 25–30.



KON-WOO KWON (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Korea University, Seoul, South Korea, in 2007 and 2009, respectively, and the Ph.D. degree in electrical and computer engineering from Purdue University, West Lafayette, IN, USA, in 2015. He was a Component Design Engineer with Intel Corporation, from 2015 to 2018. He was also with Hyundai Motor Company, Seoul, as a Research Engineer, from 2009 to 2010. He is currently an

Associate Professor with the Department of Computer Engineering, Hongik University, Seoul. His research interests include low power circuit and scalable architecture design.



KWANGOK KIM received the M.S. degree in electronic engineering from Chonnam National University, Gwangju, Republic of Korea, in 2010, and the Ph.D. degree in electronic engineering from Chungnam National University, Daejeon, Republic of Korea, in 2014. Since 2001, he has been with the Electronics and Telecommunication Research Institute, Daejeon, where he is currently a Principal Researcher. His current research interests include the next-generation

PON technologies, optical access networks, and wired/wireless converged networks.



HWANSEOK CHUNG (Senior Member, IEEE) received the Ph.D. degree in electronics engineering from Korea Advanced Institute of Science and Technology (KAIST), Daejeon, Republic of Korea, in 2003. In 2003, he was a Postdoctoral Research Associate with KAIST, where he worked on a hybrid CWDM/DWDM system for metro-area networks. From 2004 to 2005, he was with KDDI Research and Development Laboratories Inc., Saitama, Japan, and was involved in research

on wavelength converter and regenerator. Since 2005, he has been with the Electronics and Telecommunication Research Institute, Daejeon. His current research interests include high capacity and low latency optical access for mobile traffic. He is a Senior Member of the IEEE Photonics Society. He served as a Technical Committee Member of the OFC, ECOC, Optoelectronics and Communications Conference (OECC), MWP, CLEO-PR, and Photonic West. He received the Prime Minister Award from Korean Government for Distinguished Achievement in Industry, in 2019, and the Outstanding Achievement-20 in ICT Award from the Minister of Science and Technology, in 2019. He was a recipient of the Best Paper Awards and Achievement Awards from ETRI and International Conferences.



KYEONGHWAN DOO received the B.S. and M.S. degrees in electronic engineering from Chonbuk National University, Jeonju, Republic of Korea, in 1996 and 1998, respectively, and the Ph.D. degree in electronic engineering from Chungnam National University, Daejeon, Republic of Korea, in 2013. Since 2000, he has been with the Electronics and Telecommunications Research Institute, Daejeon, where he is currently a Principal Researcher. His research interests include

low-latency scheduling algorithm in high-speed networks and passive optical networks.



JEONG WOO LEE (Member, IEEE) received the B.S. and M.S. degrees in electrical engineering from Seoul National University, Seoul, South Korea, in 1994 and 1996, respectively, and the Ph.D. degree in electrical engineering from the University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 2003.

From 2003 to 2004, he was a Postdoctoral Research Associate with the Coordinated Science Laboratory, Urbana, IL, USA. From 2017 to 2018, he was a Visiting Scholar with UC San Diego, San Diego, CA, USA. Since 2004, he has been a Professor with the School of Electrical and Electronics Engineering, Chung-Ang University, Seoul. His research interests include wireless communications, coding and information theory, passive optical networks, and AI-aided communications. From 2021 to 2022, he was a Treasurer of the IEEE Seoul Section. From 2016 to 2020, he was an Associate Editor of *IET Electronics Letters*.

...